

Assignment 1: Trajectory generation for Pick and Place tasks



Matthieu DELIN Robotics and Computer Vision 2025
University of Southern Denmark (SDU)
Odense
Date of submission: 21.12.2025

1 Introduction

A scene on Mujoco is provided in Python. This scene contains objects such as box, cylinder, t.block. These objects have to be moved by the robotic arm to place behind the obstacles called "drop zone". To achieve this, 2 methods were implemented. The two methods used are "Point to Point Interpolator with a trapezoidal trajectory" and "Rapidly-exploring Random Tree (RRT)". Both methods aim to transition the robot from a starting configuration q_{start} to a goal configuration q_{goal} . However, they differ fundamentally in how they handle the path and the environment.

2 Methods

2.1 Differences and similarities

The Point-to-Point (P2P) method is primarily an interpolation technique. It connects pre-defined waypoints with a smooth time law. The collision avoidance relies entirely on the user manually selecting safe intermediate waypoints. In contrast, RRT is a path planning algorithm that automatically explores the configuration space to find a collision-free path among obstacles.

The P2P approach is deterministic, it will always produce the exact same trajectory for the same inputs. RRT is a randomized algorithm. Therefore, different runs may produce different paths, and the computation time can vary.

The P2P method with a trapezoidal profile guarantees continuous velocity and bounded acceleration, resulting in smooth motion. The raw RRT output is a series of straight lines in joint space, which can result in jerky motion if not post-processed or smoothed.

2.2 Analysis of Position, Velocity and Acceleration profiles

- **Point to point with a trapezoidal profile:** The joint position follows a parabolic curve during acceleration/deceleration phases and a linear slope during the constant velocity phase. This ensures a smooth path in the joint space. The velocity profile is trapezoidal. It increases linearly from zero to a maximum cruise velocity (v_{max}), remains constant, and then decreases linearly to zero. This continuity prevents mechanical shocks. The acceleration is piecewise constant (positive during start, zero during cruise, negative during stop). (Figure:4)
- **Rapidly-exploring Random Tree:** The generated path consists of linear segments connecting random states (nodes) in the configuration space. The robot moves in a straight line from one node to the next. On each segment, the velocity is constant. However, at the nodes (waypoints), the velocity direction changes instantaneously, leading to discontinuities. Theoretically, the acceleration is zero along the segments and infinite (Dirac delta) at the nodes where direction changes sharply. In the implementation, the robot controller ("*move_j_via*") acts as a low-level smoother to physically execute these transitions, but the generated path itself is not dynamically optimal. (Figure:1)

Algorithm 1 Pick and Place using Point-to-Point Trapezoidal Interpolation

Require: q_{start} (Current Configuration), $Object_{pose}$, $Drop_{pose}$ **Ensure:** $Trajectory$ (List of joint configurations)

```

1:  $W \leftarrow [q_{start}]$  ▷ Initialize list of Waypoints
2: Step 1: Define Key Waypoints (Manual Planning)
3:  $q_{grasp} \leftarrow \text{InverseKinematics}(Object_{pose})$ 
4:  $q_{approach} \leftarrow \text{InverseKinematics}(Object_{pose} + z\_offset)$ 
5:  $q_{safe} \leftarrow \text{PredefinedSafeJoints}$  ▷ To avoid obstacles
6:  $q_{drop} \leftarrow \text{InverseKinematics}(Drop_{pose})$ 
7: Append Sequence to W:
8:  $W.append(q_{approach}, q_{grasp}, q_{approach}, q_{safe}, q_{drop})$ 
9: Step 2: Generate Trajectory
10:  $Trajectory \leftarrow []$ 
11: for  $i \leftarrow 0$  to  $\text{length}(W) - 2$  do
12:    $q_{current} \leftarrow W[i]$ 
13:    $q_{next} \leftarrow W[i + 1]$ 
14:    $T \leftarrow \text{Duration based on distance}(q_{current}, q_{next})$ 
15:   ▷ Interpolate between points using Trapezoidal Profile
16:    $Segment \leftarrow \text{TrapezoidalInterp}(q_{current}, q_{next}, T)$ 
17:    $Trajectory.append(Segment)$ 
18: end for
19: return  $Trajectory$ 

```

Algorithm 2 Path Planning using RRT (OMPL Implementation)

Require: $q_{start}, q_{goal}, \text{Environment}$ (Obstacles)**Ensure:** Path (Sequence of collision-free configurations)

```

1:  $Cspace \leftarrow \text{RealVectorStateSpace}(6)$  ▷ Define 6-DOF Joint Space
2:  $\text{SetBounds}(Cspace, [,])$ 
3:  $SS \leftarrow \text{SimpleSetup}(Cspace)$ 
4: Setup Validity Checker:
5: function  $\text{ISVALID}(q)$ 
6:    $\text{UpdateMuJoCoModel}(q)$ 
7:   if  $\text{CollisionDetected}(\text{Environment})$  then return False
8:   elsereturn True
9:   end if
10: end function
11:  $SS.setValidityChecker(\text{isValid})$ 
12:  $SS.setStartAndGoalStates(q_{start}, q_{goal})$ 
13:  $SS.setPlanner(\text{RRT})$ 
14: Execute Planner:  $\text{State solved} \leftarrow SS.solve(\text{timeout}=10.0s)$ 
15: if  $\text{solved}$  then
16:    $\text{RawPath} \leftarrow SS.getSolutionPath()$ 
17:    $Trajectory \leftarrow []$ 
18:   for  $\text{state}$  in  $\text{RawPath}$  do
19:      $Trajectory.append(\text{extract\_joint\_values}(\text{state}))$ 
20:     return  $Trajectory$ 
21:   else
22:     return Failure
23:   end if

```

3 Results and Discussion

The performance of the two implemented methods, Point-to-Point Interpolation (PIP) with a trapezoidal profile and Rapidly-exploring Random Tree (RRT), was evaluated based on trajectory smoothness, execution time, and robustness. The graphical results are presented in Figures 1 to 2.

3.0.1 Trajectory and Kinematics Analysis

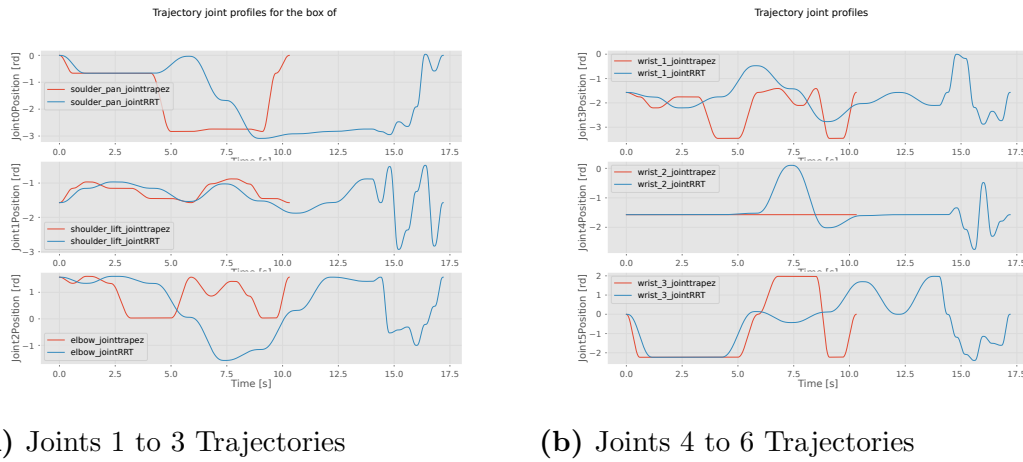


Figure 1 Comparison of Joint Trajectories: Trapezoidal (PIP) vs. RRT

The figure above compares the joint positions over time. The PIP method (represented by the smoother curves) clearly follows a deterministic path. In contrast, the RRT method generates a path that is significantly longer and contains redundant movements ("useless points"). This is inherent to the random sampling nature of the algorithm, which explores the configuration space stochastically until a solution is found.

The Direct Kinematics plot (Figure 2) visualizes the Cartesian path of the end-effector of each axis as a function of time. The PIP method exhibits a logical "pick-and-lift" motion pattern. The RRT path, however, wanders within the workspace. While it succeeds in avoiding obstacles, the path is far from optimal and results in a longer execution time compared to the trapezoidal interpolator.

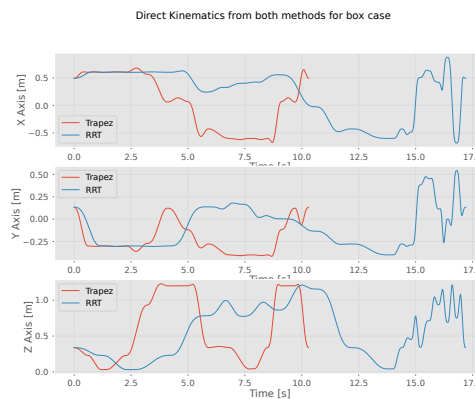


Figure 2 Direct Kinematics (End-Effector Path) for both methods

3.0.2 Velocity and Acceleration Profiles

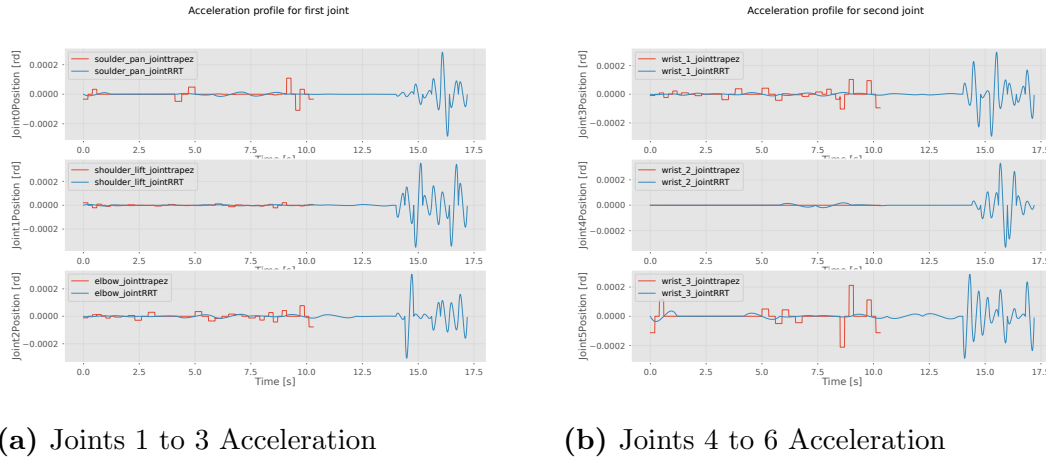


Figure 3 Acceleration profiles comparison

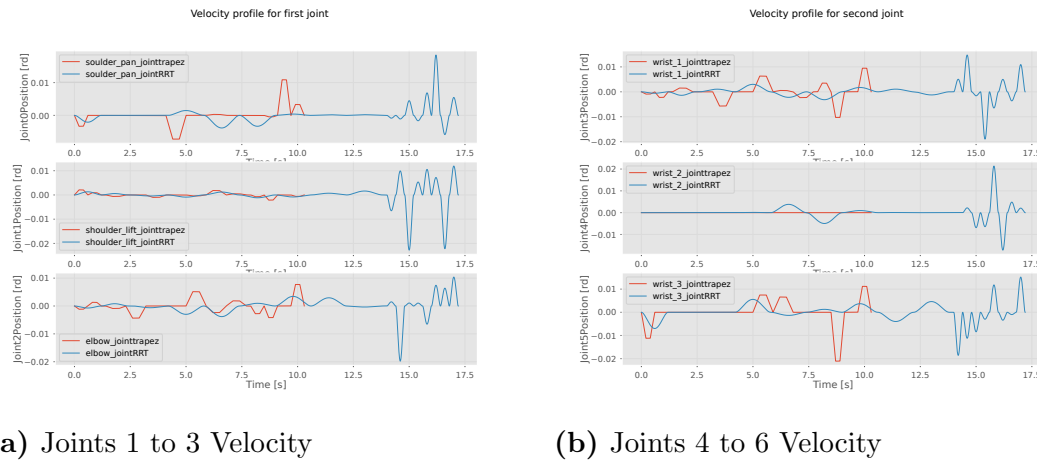


Figure 4 Velocity profiles comparison

Figure 4 and 3 highlight the dynamic differences:

- **PIP:** The velocity profiles clearly show the trapezoidal shape (linear increase, constant speed, linear decrease), respecting the acceleration limits. The motion is smooth and controlled.
- **RRT:** The profiles are erratic with abrupt changes. Since standard RRT does not inherently consider velocity constraints during the planning phase, the resulting trajectory requires the controller to make sharp adjustments, leading to a less stable motion.

3.0.3 Robustness and Reliability

In terms of reliability, the PIP method proved to be more robust. During the manipulation of the cylinder, we observed that the RRT algorithm occasionally failed. The randomized path sometimes brought the robot too close to obstacles

or generated jerky movements that caused the object to slip from the gripper. The trapezoidal interpolator, being less abrupt and fully defined by the user's waypoints, maintained a stable grasp throughout the task.

4 Conclusion

In this assignment, we implemented and compared a deterministic trajectory generator (Point-to-Point with Trapezoidal profile) and a probabilistic path planner (RRT) for a pick-and-place task.

The results demonstrate that for a structured environment with known obstacles, the Point-to-Point Interpolator is superior in terms of efficiency, smoothness, and cycle time. The RRT algorithm, while powerful for finding solutions in complex clutter without manual waypoint definition, produces suboptimal paths that are longer and jerkier.

Future Improvements To improve the RRT performance, path smoothing techniques (such as "shortcutting") could be applied to remove redundant nodes. It is worth noting that we attempted to implement a filter to prune these useless points. However, this post-processing increased the failure rate, as the smoothed paths occasionally clipped obstacles or violated the strict orientation constraints required for holding the object. Therefore, the raw RRT path was retained for safety, despite its inefficiency. A potential improvement would be to implement RRT that take care of the size of the object grasped.